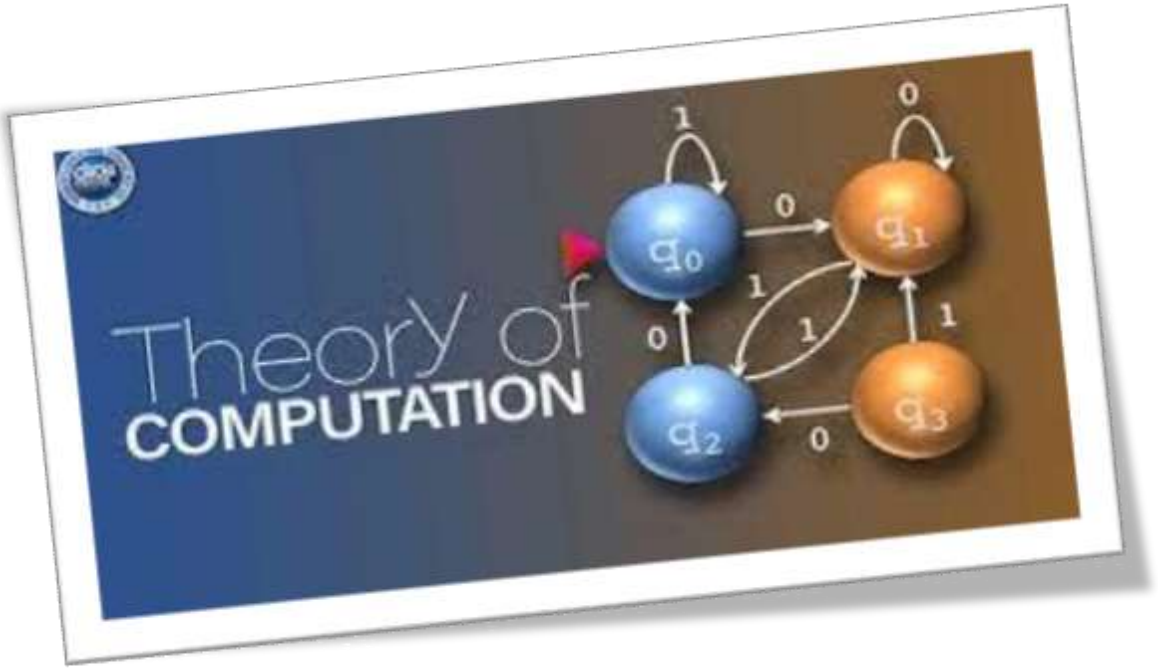
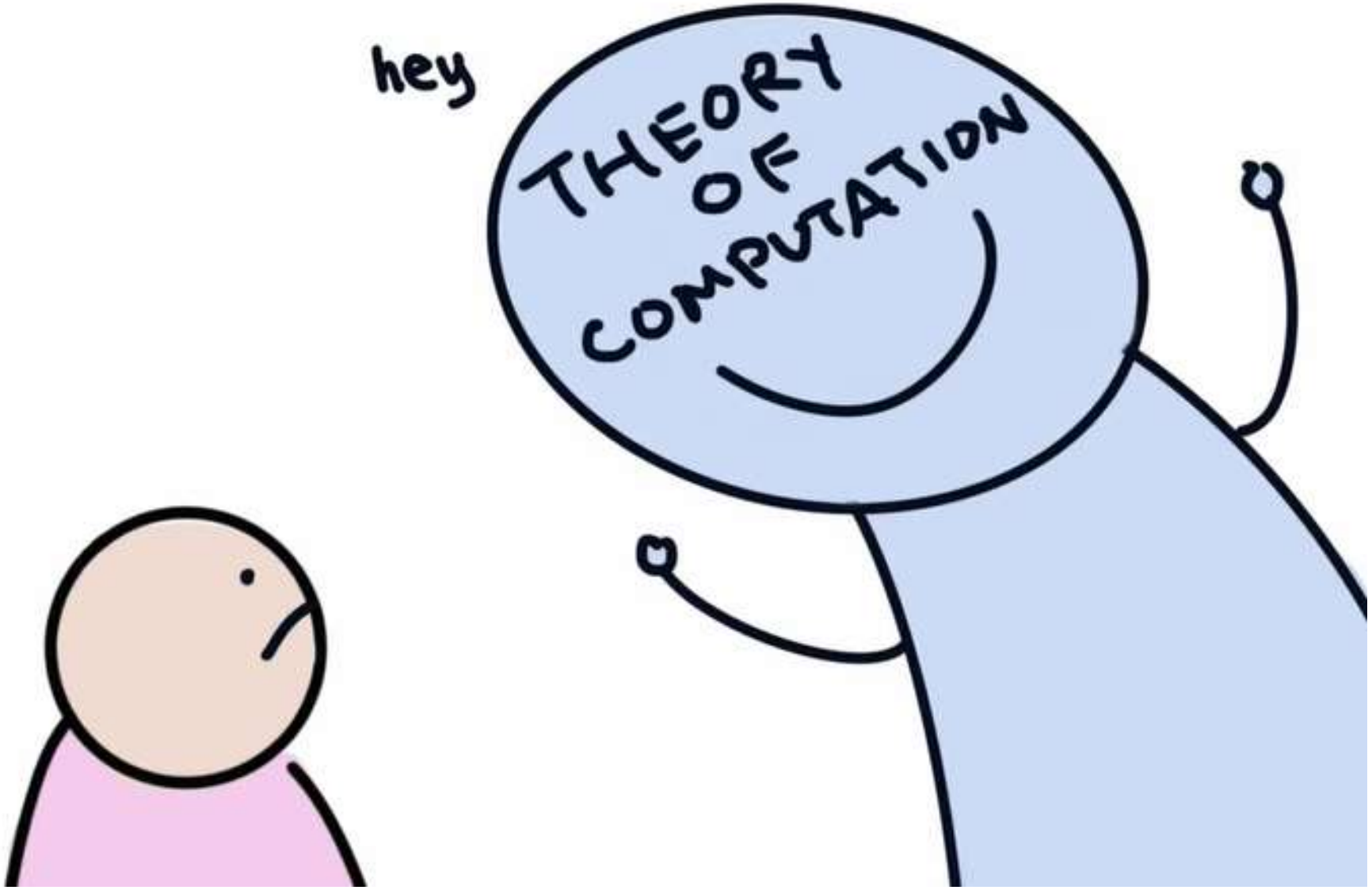
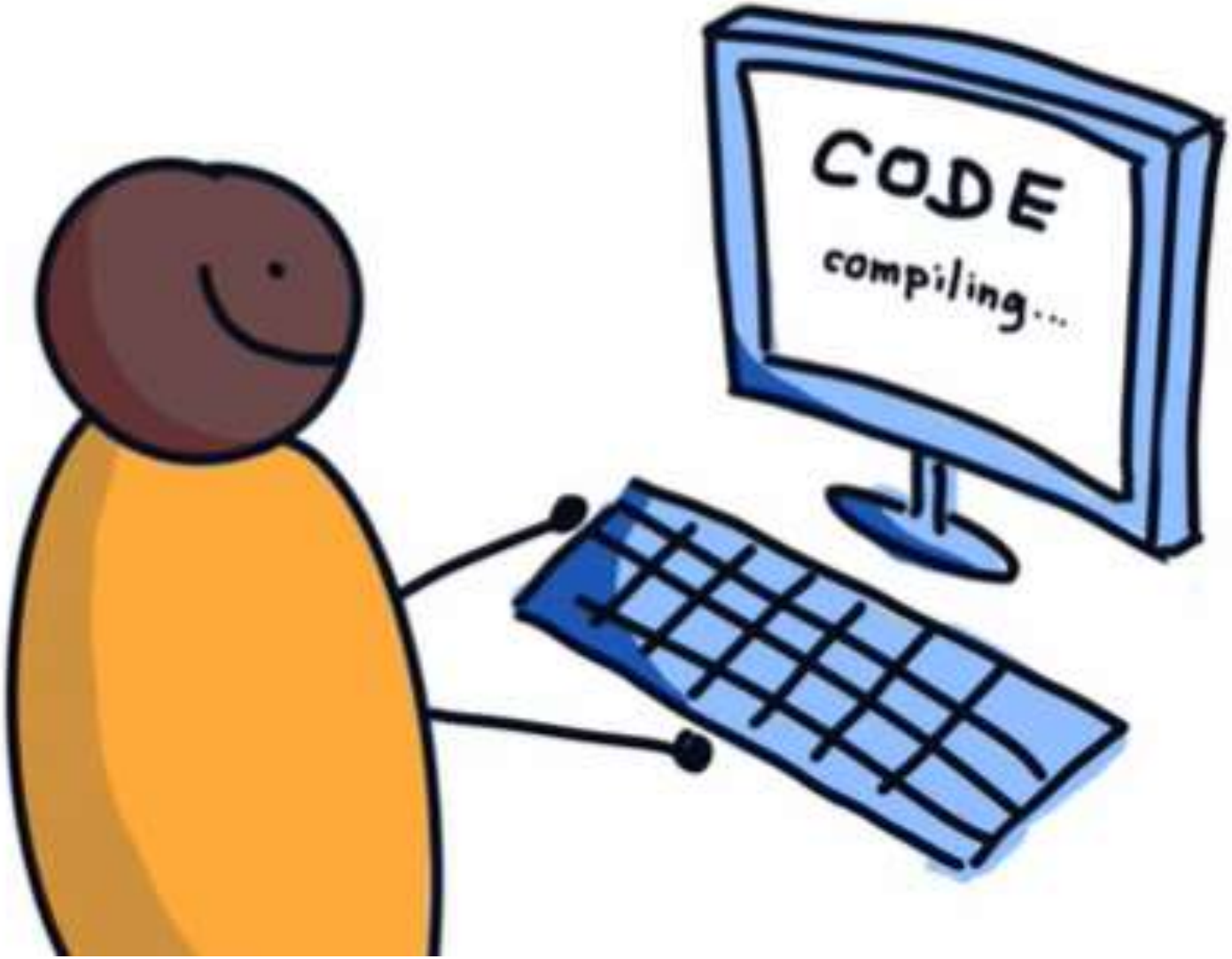
BCSE304L

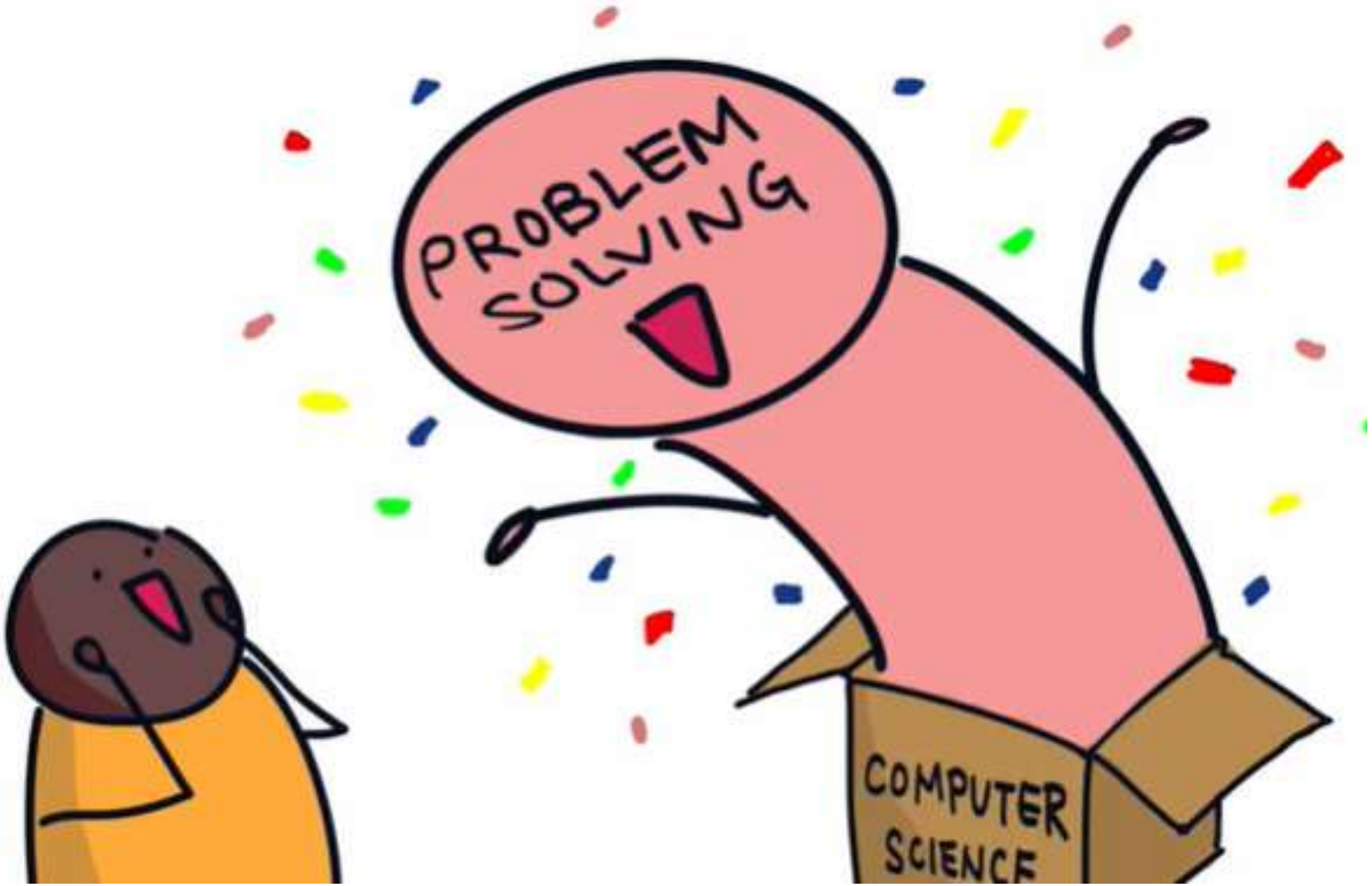
Theory of Computation



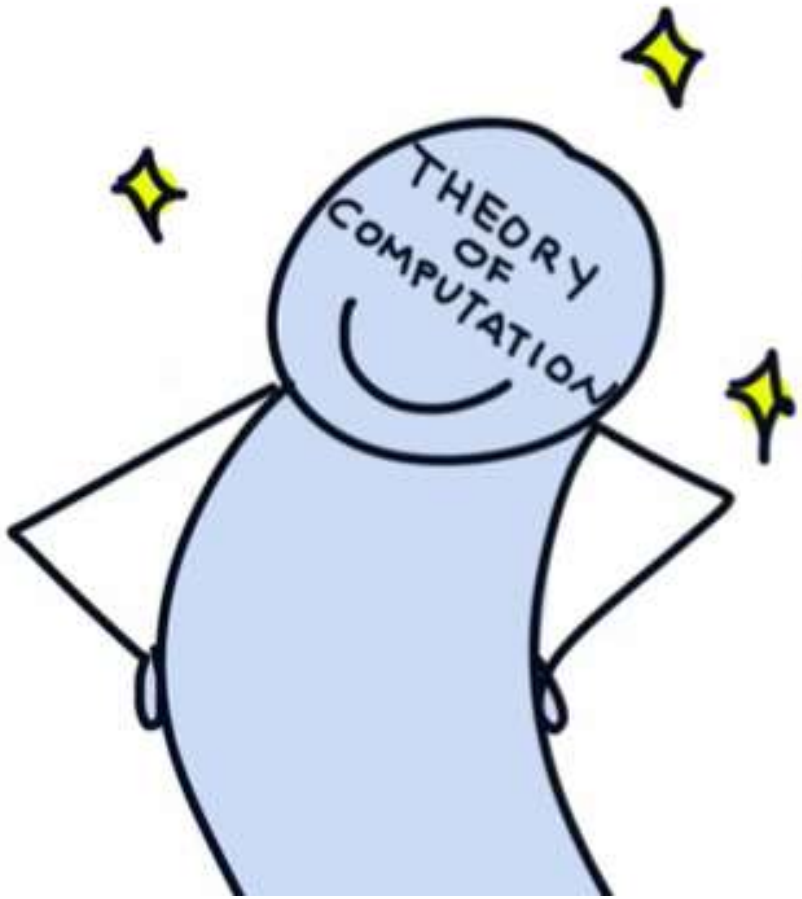

hey
THEORY OF COMPUTATION


CODE
compiling...

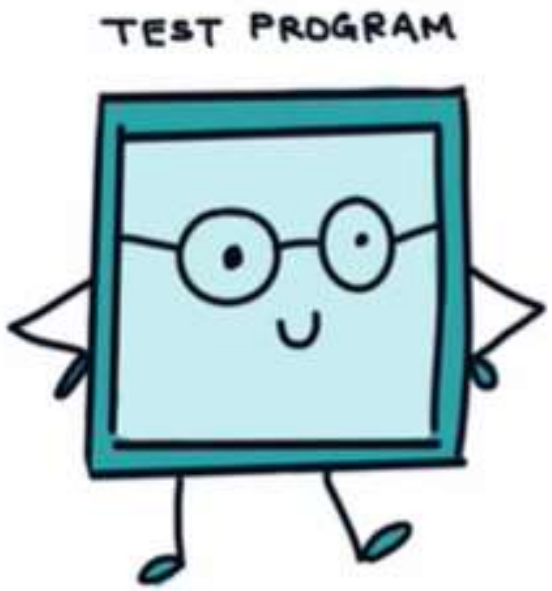
ERROR


PROBLEM SOLVING
COMPUTER SCIENCE

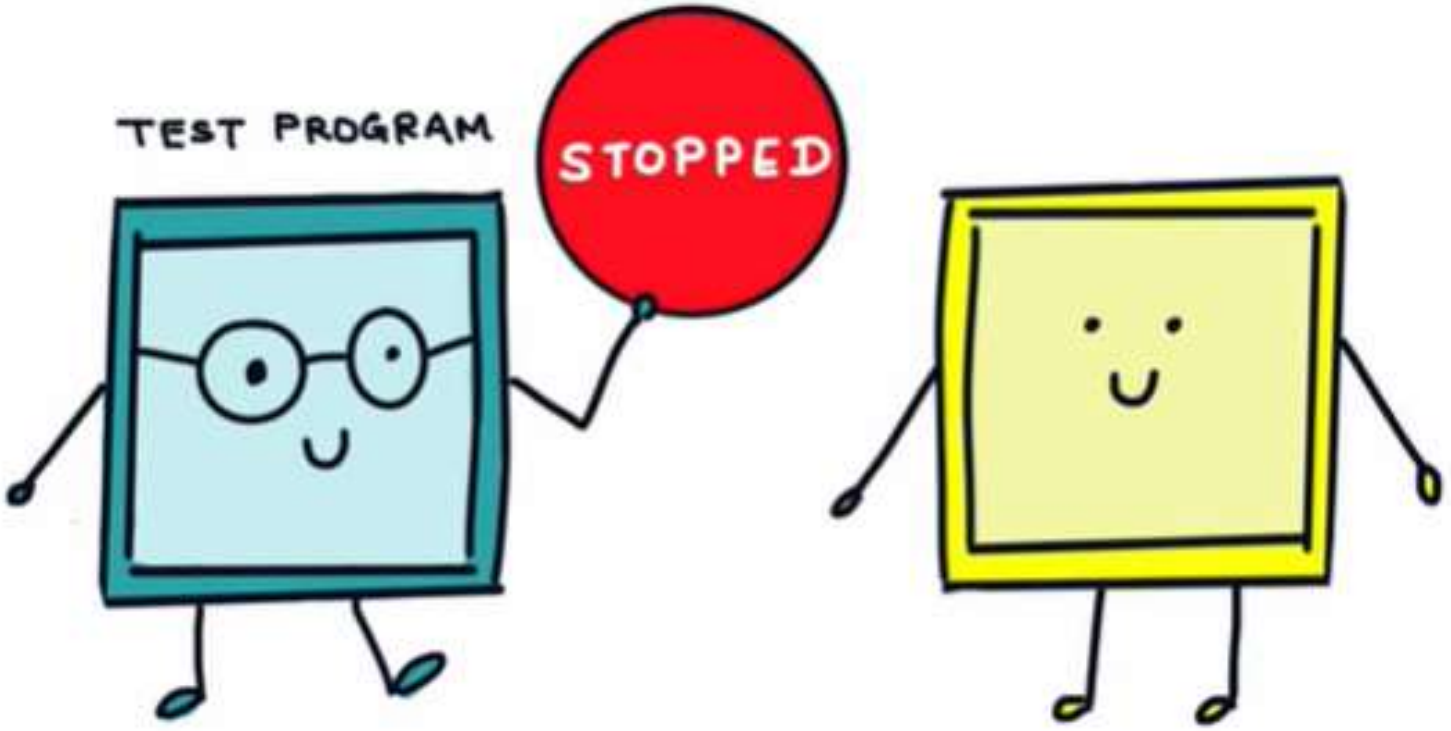
THEORY
OF
COMPUTATION



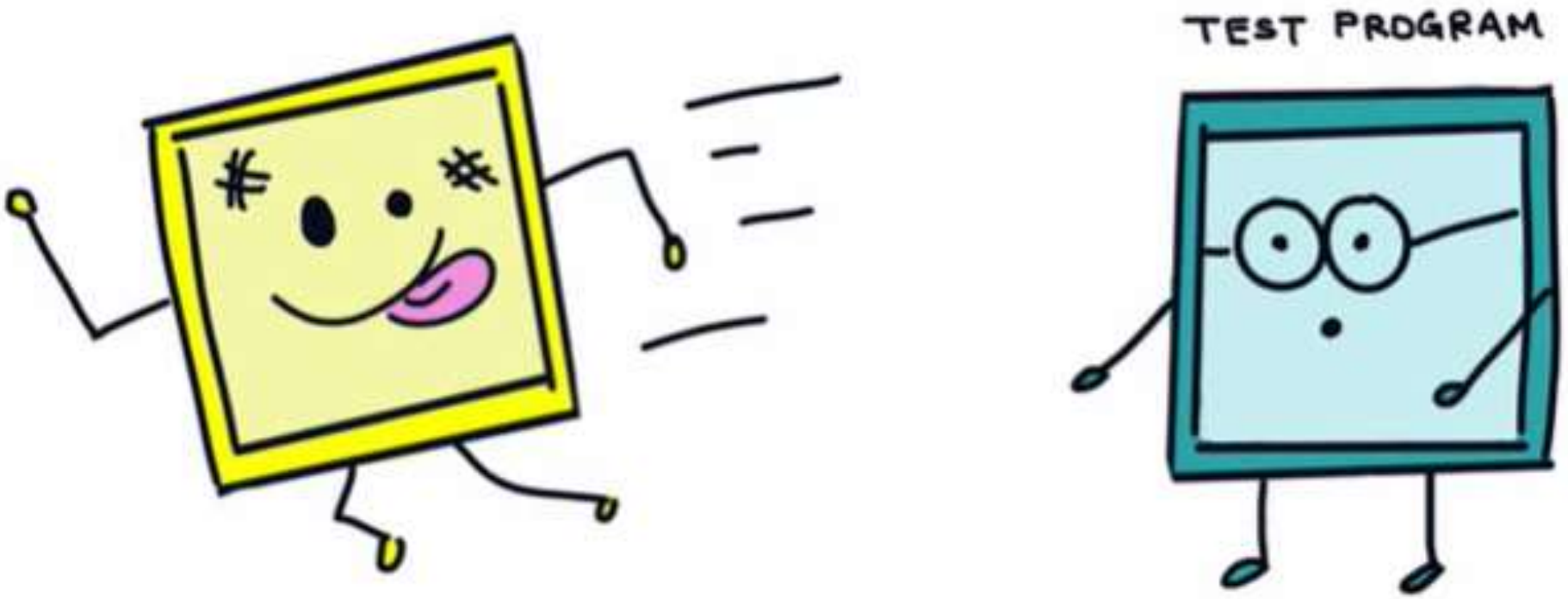
WHAT ARE THE FUNDAMENTAL CAPABILITIES & LIMITATIONS OF COMPUTERS?



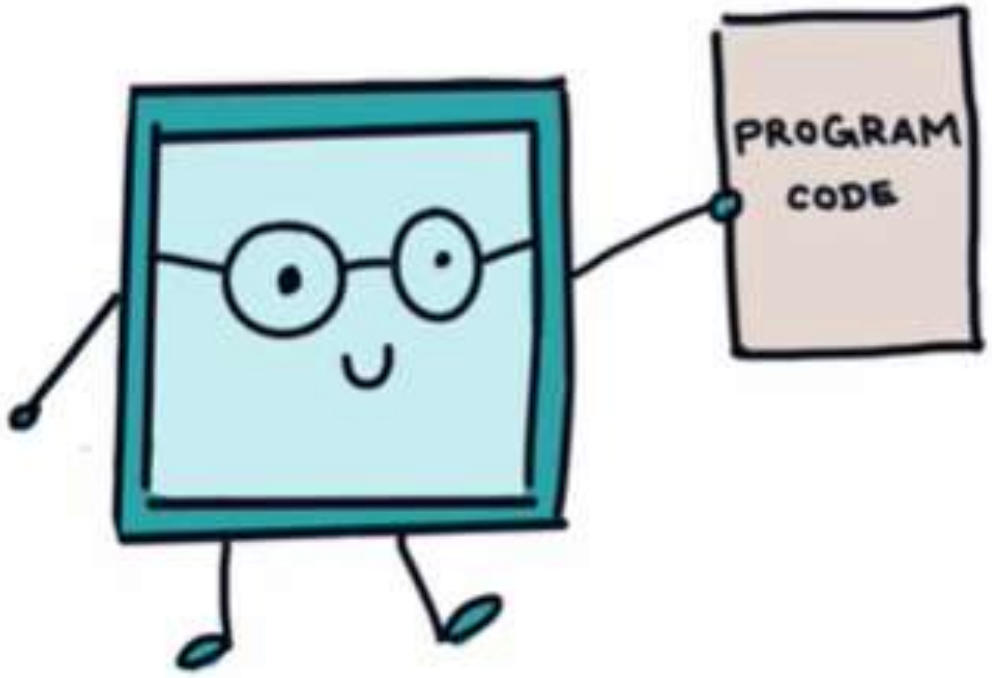
Automated Test of the program. Check for the each and every input



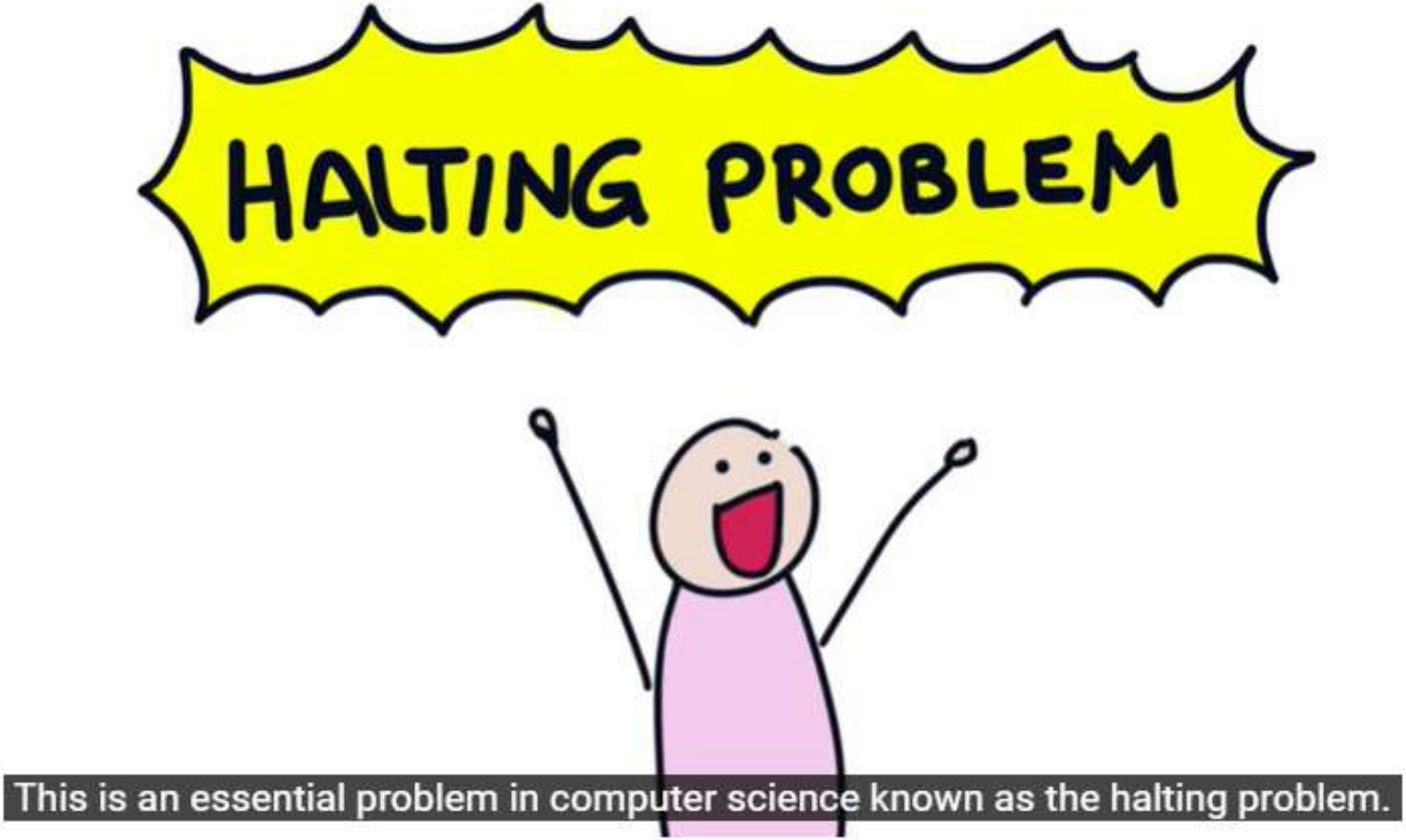
Finish running and stop



However, our automated test that we're going to write never actually runs the program. It just reads its lines of code



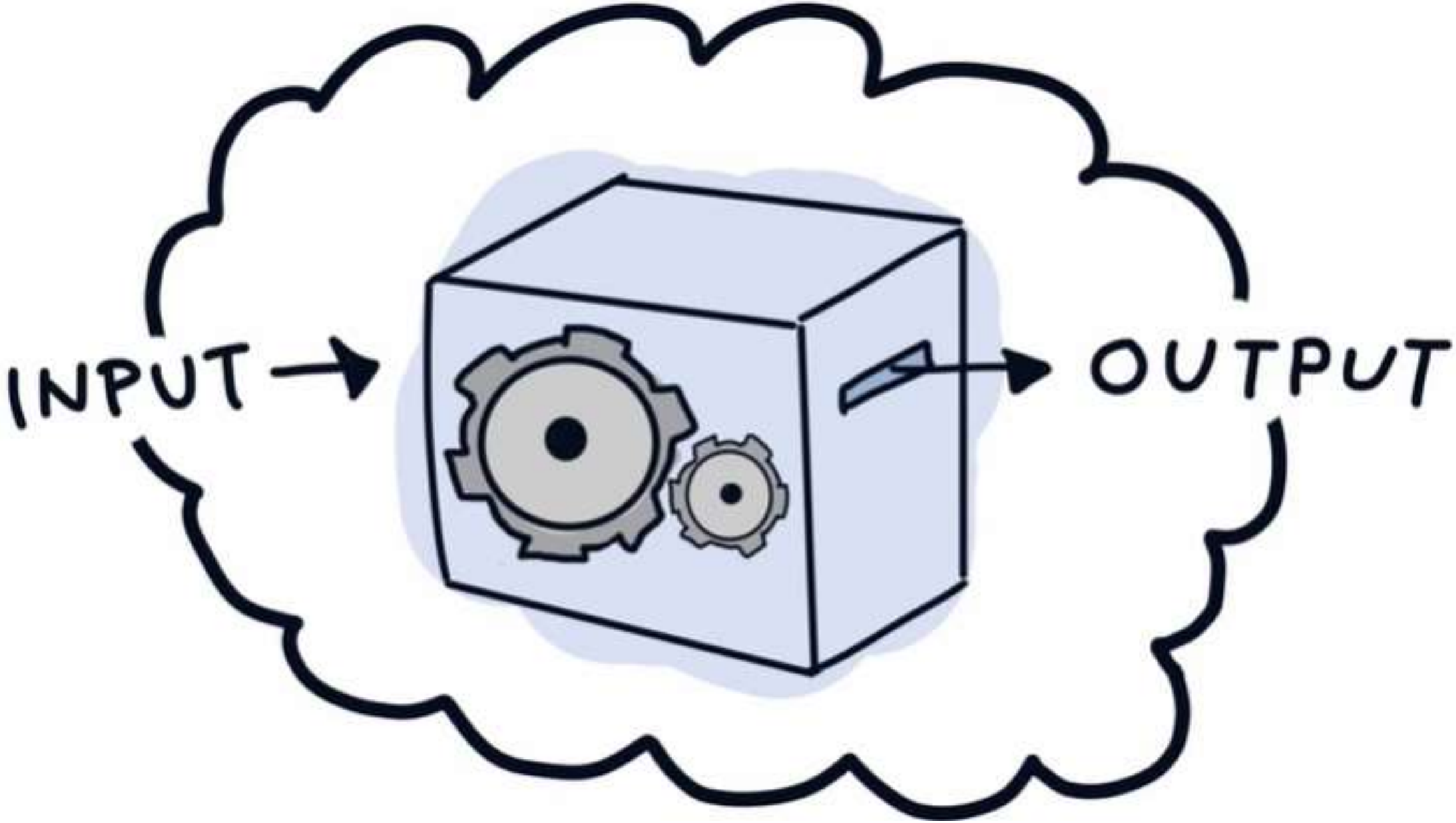
Do you think it is possible to test the code without actually executing it?


HALTING PROBLEM
This is an essential problem in computer science known as the halting problem.

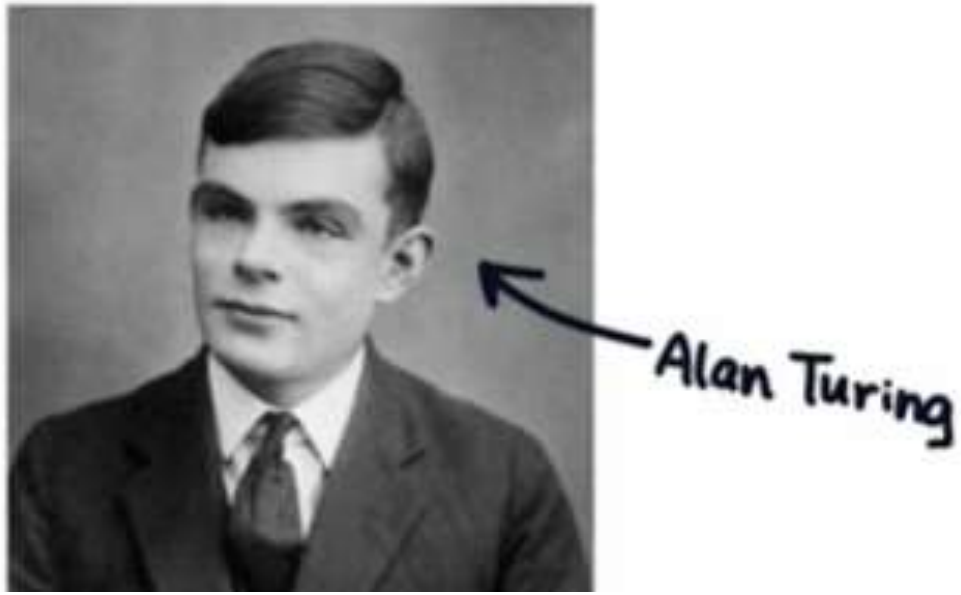
The halting problem is a decision problem in the theory of computation that asks if a computer program will eventually stop or run forever when given a specific input.

We want to prove mathematically why the problem is not been solved.

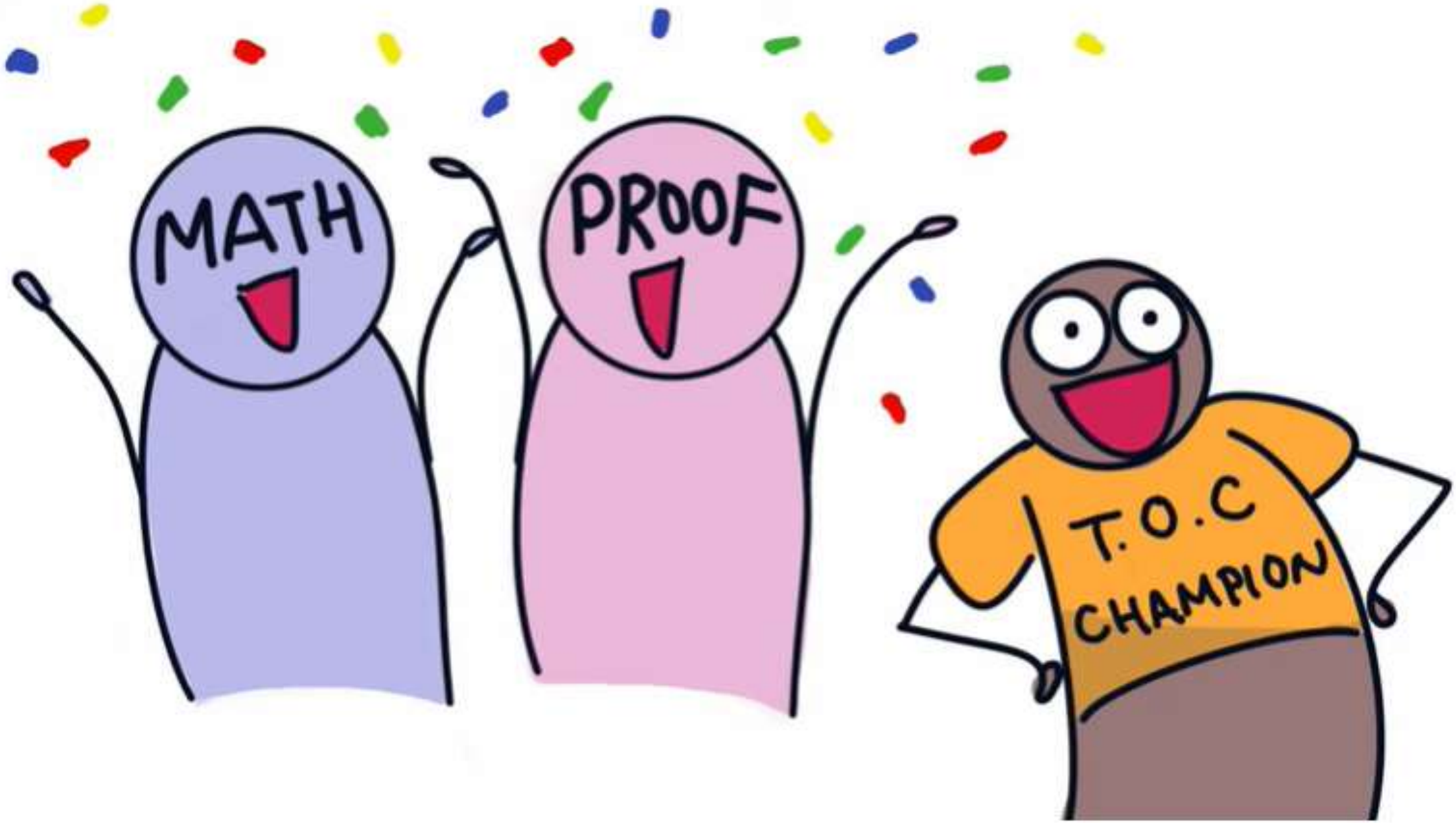
MODELS OF COMPUTATION

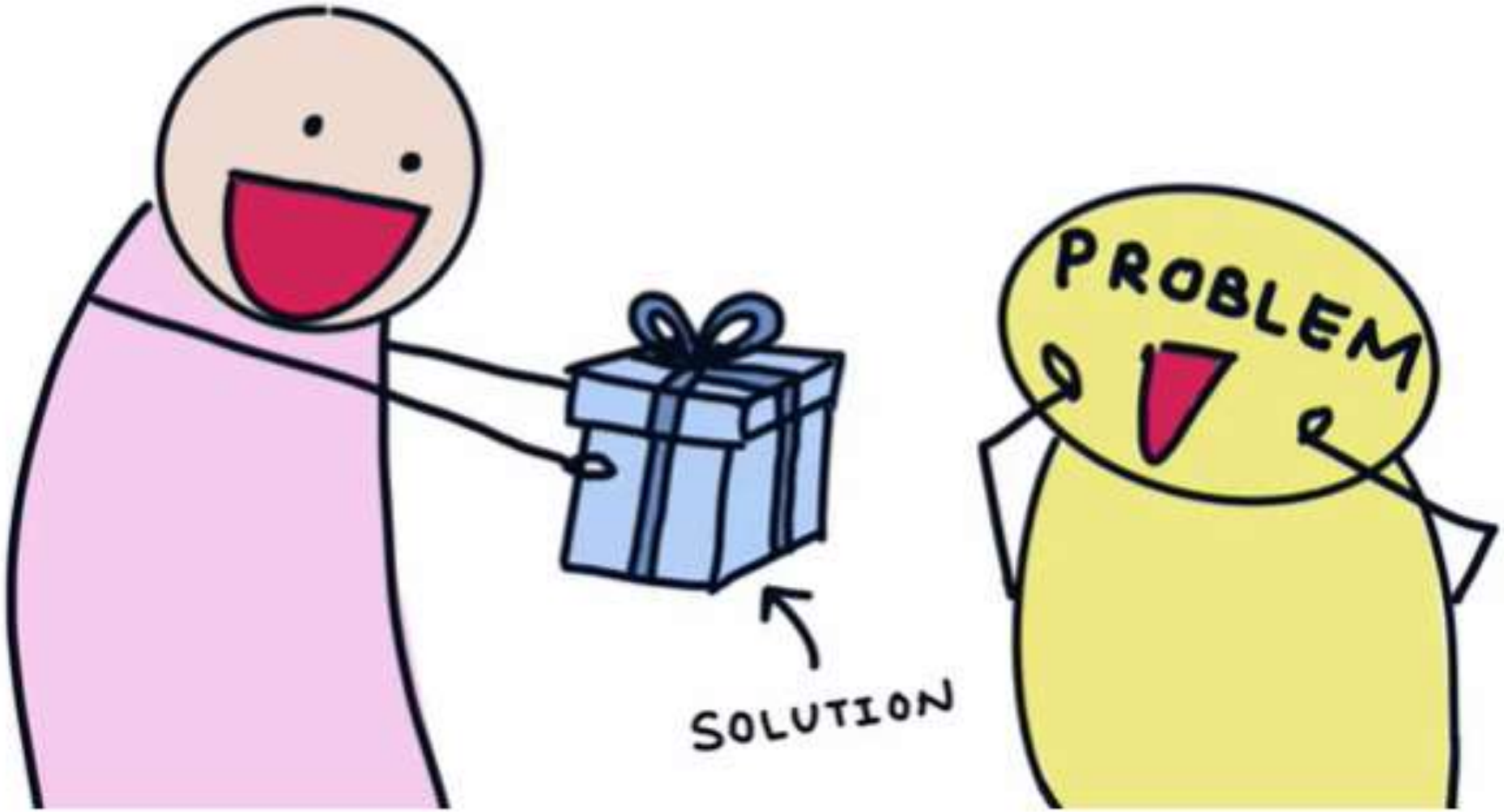

INPUT
OUTPUT

TURING MACHINE



A Turing machine (TM) is a **mathematical model** of a **computer** that **reads** and **writes** to an **infinite tape to perform computations**. It's a **simple model** that can **implement any computer algorithm**


MATH
PROOF
T.O.C
CHAMPION


PROBLEM
SOLUTION

Module: 1

Introduction to Languages and Grammars

Contents

1. Recall on Proof techniques in Mathematics
    a. Introduction to Formal Proof
    b. Additional Forms of Proof
    c. Proof by Contradiction
    d. Inductive Proofs

2. Overview of a Computational Models
3. Languages and Grammars
4. Alphabets, Strings and Operations on Languages
5. Overview on Automata

Theory of Computing

- Most fundamental course of Computer Science
- Will help you understand how people have taught about Computer Science as Science in the past 50-60 years.
- To know the foundations and principles of computer science
- Branch that deals with how efficiently problems can be solved on a model of computation, using an algorithm
- Branches:
  - Automata Theory
  - Computability Theory
  - Computational Complexity Theory

Example

1. Design a machine that accepts all binary strings that ends in 0
   - Ex: 1101010, 100001

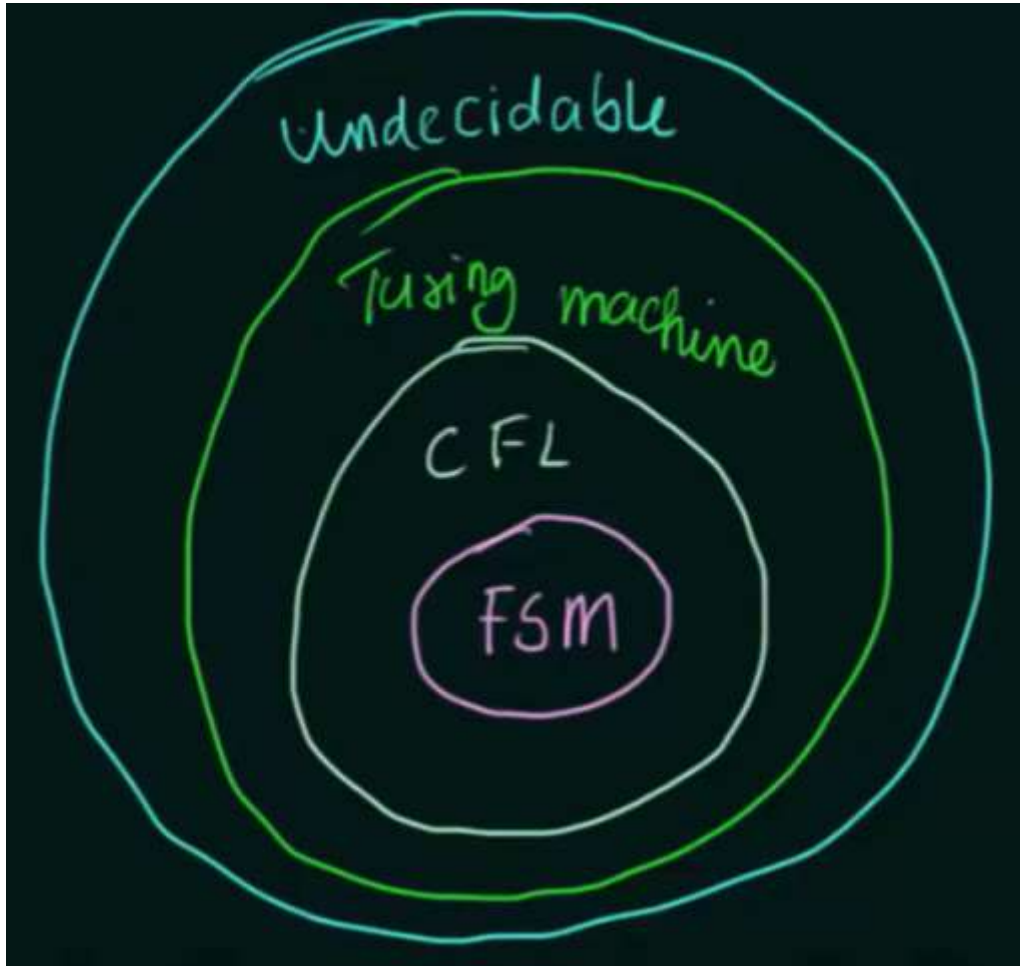
2. Design a machine that accepts all valid Java codes.
   - Ex: Compilers

3. Design a machine that accepts all valid Java codes and does not go into an infinite loop.

Applications

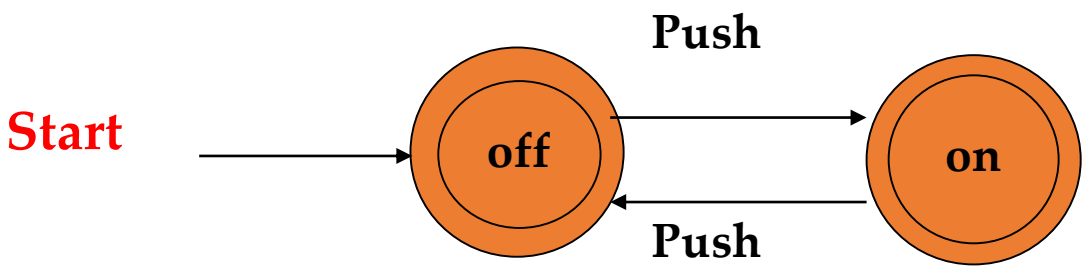
1. Traffic Lights
2. Lifts and elevators
3. Cryptography
4. Design and Analysis of Algorithm
5. Quantum Calculation
6. Logic within Computer Science
7. Randomness within Calculation

Cont…



Automata Theory

- Study of abstract computing machine as well as computational problems solved with them
- Study of mathematical properties of automata
- Ex: FA modeling of on/off switch

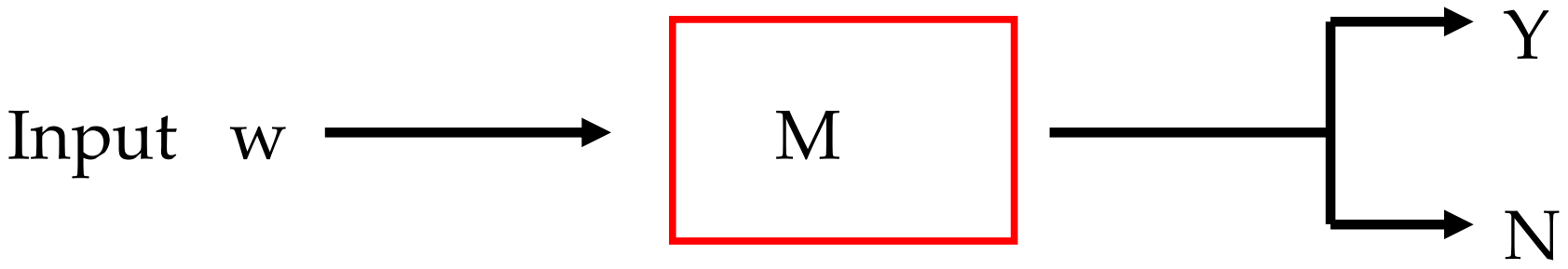


- Need for Automata Theory
  - Essential for the study of the limits of computation
  - Pattern searching in websites
  - Designing and checking the behaviour of the digital circuits

Overview of a computational model

- Computation

  Program running in computer



Central Concepts of Automata Theory

- Alphabets (Σ)
  - A finite, non-empty set of symbols
  - Ex:
    a) $\Sigma = \{0, 1\}$, the binary alphabet
    b) $\Sigma = \{a, b, \ldots, z\}$, the set of all lower-case letter
    c) The set of all ASCII characters

Central Concepts of Automata Theory – cont..

- Strings
  - A string (or word) is a finite sequence of symbols chosen from some alphabet
  - Example
    - 1011 is a string from alphabet $\Sigma = \{0, 1\}$
  - Empty string, ε → a string with zero occurrences of symbols
  - Length $|w|$ of string w → the number of positions for symbols in w
    - Examples: $|0111| = 4$, $|\varepsilon| = 0$, …

Central Concepts of Automata Theory – cont..

- Power of an alphabet, Σ^k
  - A set of all strings of length k
- Examples
  - Given $\Sigma = \{0, 1\}$, we have

    $\Sigma^0 = \{\varepsilon\}$, $\Sigma^2 = \{00, 01, 10, 11\}$
  - Set of all strings over Σ → denoted as Σ^*

    $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \ldots$

- Set of nonempty strings from Σ → Σ^+

  $\Sigma^+ = \Sigma^* - \{\varepsilon\}$

  $\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \ldots$

  $\Sigma^* = \Sigma^+ \cup \{\varepsilon\}$

Central Concepts of Automata Theory – cont..

- Concatenation of two strings x and y is xy
  - Example
    - If $x = 01101$, $y = 110$,
    - then $xy = 01101110$, and $xx = x^2 = 0110101101$, …
- ε is the identity for concatenation

  since $\varepsilon w = w\varepsilon = w$

Central Concepts of Automata Theory – cont..

- Languages
  - A language is a set of strings all chosen from some Σ^*
  - If Σ is an alphabet, and $L \subseteq \Sigma^*$, then L is a language over Σ.
  - **Examples**
    - The set of all legal English words is a language ➔ {a, b, c, …., z}
    - The set of all strings of n 0's followed by n 1's for $n \geq 0$ ➔ {ε, 01, 0011, 000111, …}

Central Concepts of Automata Theory – cont..

- Languages – *cont…*
  - The set of strings of equal no. of 0's and 1's ➔ {ε, 01, 10, 0011, 0101,…}
  - Σ^* is an infinite language for any alphabet Σ
  - ϕ = the empty language (no string or not the empty string, ε) is a language over any alphabet
  - $\phi \neq \{\varepsilon\}$

Languages – *cont…*

- Σ^* denotes the set of all finite strings over Σ
- A formal language L over the alphabet Σ is a subset of Σ^*
- That is L is a subset of Σ^*
  - Let $\Sigma = \{0, 1\}$ be the alphabet.
  - $L = \{w \in \Sigma^* \mid$ the number of 1's in w is even$\}$.
  - ε, 0, 00, 11, 000, 110, 101, 011, 0000, 1100, 1010, 1001, 0110, 0101, 0011, … are in L

Properties of languages

1. Concatenation
2. Substring
3. Reversal
4. Kleene Closure and Positive Closure

Concatenation of languages

- The concatenation of two languages L_1 and L_2 is given by,

$$L = L_1 L_2 = \{w \in \Sigma^* ; w = xy \text{ for some } x \in L_1 \text{ and } y \in L_2\}$$

- Concatenation of two strings w_1 and w_2 is defined as the sequence of symbols in w_1 followed by sequence of symbols in w_2

- Let $w_1 = abcd$ and $w_2 = fgh$

Then

$$w_1 w_2 = abcdfgh$$
$$w_2 w_1 = fghabcd$$

The concatenation of string x for n times, where $n \geq 0$, is denoted by x^n

$x^0 = \varepsilon$, $x^1 = x$, $x^2 = x\,x$,

$x^3 = x\,x\,x$

…

Substring

Let x and y be strings over an alphabet, Σ

The string x is a substring of y if there exist strings w and z over Σ such that $y = w\,x\,z$.

- ε is a substring of every string.
- For every string x, x is a substring of x itself.

Example

- ε, *comp* and *computation* are substrings of *computation*.

Reversal

The reversal of a string is a string with same symbols in reverse order, denoted by w^R

Example

Let $w = abcdfgh$

Then

$w^R = hgfdcba$

Kleen Star

- The Kleen star of a language L is denoted by L* which is the set of all strings obtained by concatenating zero or more strings from L

$$L^* = \{w \in \Sigma^* ; w = w_1 \ldots w_k \text{ for } k>=0 \text{ and } w_1, w_2, \ldots, w_k \in L\}$$

Let $\sum = \{a, b\}$ i.e. $\sum^1 = \{a, b\}$

$\sum^2 = \sum^1 . \sum^1 = \{a, b\}. \{a, b\} = \{aa, ab, ba, bb\}$

$\sum^3 = \sum^1 . \sum^1 . \sum^1 = \{a, b\}. \{a, b\}. \{a, b\}$

$= \{aa, ab, ba, bb\}. \{a, b\}$

$= \{aaa, aab, aba, abb, baa, bab, bba, bbb\}$

… etc

Kleen Star

$\sum^0 = \{\varepsilon\}$

The **Kleene closure** of $\sum$, denoted by $\sum^*$ is defined as follows:
$\sum^* = \sum^0 \cup \sum^1 \cup \sum^2 \cup \sum^3 \ldots\ldots\ldots = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, \ldots.\}$

The **Positive closure** of Σ, denoted by $\sum^+$ is defined as follows:
$\sum^+ = \sum^* - \{ \varepsilon \}$

Operations on Languages

- Complementation
- Union
- Intersection
- Concatenation
- Reversal
- Closure

Complementation

Let L be a language over an alphabet, Σ.

The complementation of L, denoted by $\overline{L}$ (or L'), is $\Sigma^* - L$.

Example:

Let $\Sigma = \{0, 1\}$ be the alphabet.

$L = \{\omega \in \Sigma^* \mid$ the number of 1's in ω is even$\}$.

$L' = \{\omega \in \Sigma^* \mid$ the number of 1's in ω is not even$\}$.

Union

Let L_1 and L_2 be languages over an alphabet Σ.

The union of L_1 and L_2, denoted by $L_1 \cup L_2$, is

$L_1 \cup L_2 = \{x \mid x \text{ is in } L_1 \text{ or } L_2\}$.

Example:

$\{x \in \{0,1\}^* \mid x \text{ begins with } 0\} \cup \{x \in \{0,1\}^* \mid x \text{ ends with } 0\}$

$= \{x \in \{0,1\}^* \mid x \text{ begins or ends with } 0\}$

Intersection

Let L_1 and L_2 be languages over an alphabet Σ.

The intersection of L_1 and L_2, denoted by $L_1 \cap L_2$, is

$\{ x \mid x$ is in L_1 and $L_2 \}$.

Example:

$\{ x \in \{0,1\}^* \mid x$ begins with $0\} \cap \{ x \in \{0,1\}^* \mid x$ ends with $0\}$

$= \{ x \in \{0,1\}^* \mid x$ begins and ends with $0\}$

Concatenation

Let L_1 and L_2 be languages over an alphabet Σ.

The concatenation of L_1 and L_2, denoted by $L_1 \cdot L_2$, is

$$\{w_1 \cdot w_2 \mid w_1 \text{ is in } L_1 \text{ and } w_2 \text{ is in } L_2\}.$$

Example

$L_1 = \{x \in \{0,1\}^* \mid x \text{ begins with } 0\}$

$L_2 = \{x \in \{0,1\}^* \mid x \text{ ends with } 0\}$

$L_1 \cdot L_2 = \{ x \in \{0,1\}^* \mid x \text{ begins and ends with } 0 \text{ and } length(x) \geq 2\}$

Reversal

Let L be a language over an alphabet Σ.

The reversal of L, denoted by L^R, is $\{w^R \mid w \text{ is in } L\}$.

Example

$\boldsymbol{L = \{x \in \{0,1\}^* \mid x \text{ begins with } 0\}}$

$L^R = \{x \in \{0,1\}^* \mid x \text{ ends with } 0\}$

Kleene's closure

Let L be a language over an alphabet Σ.

The Kleene's closure of L, denoted by L^*, is $\{x \mid$ for an integer $n \geq 0$, $x = x_1 x_2 \ldots x_n$ and $x_1, x_2, \ldots, x_n$ are in $L\}$.

That is, $L^* = \cup_{i=0}^{\infty} L^i$

Example: Let $\Sigma = \{0,1\}$ and

$L_e^* = \{\omega \in \Sigma^* \mid$ the number of 1's in ω is even$\}$

$(L_e)^* = \{\omega \in \Sigma^* \mid$ the number of 1's in ω is odd$\}$

Central Concepts of Automata Theory – cont…

- **Grammar**
  - A grammar of a language, G is defined as G = (V, T, S, P)
  - Where,

    V or N ➔ Finite set of objects called Variables / Non-terminals

    T ➔ Finite set of objects called Terminals

    $S \in V$ ➔ Start Symbol

    P ➔ Finite set of Productions

    Ex: G = {(S, T), (a, b), S, P}

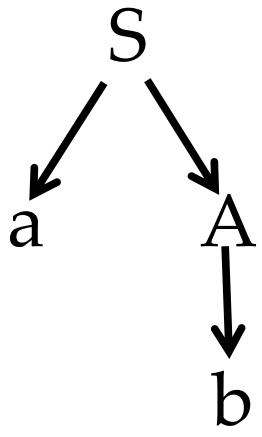
    Note: $N \cap T = \phi$

Grammar (Examples)

G = ({S, A}, {a, b}, P, S), where

P: $S \rightarrow aA$

$A \rightarrow b$



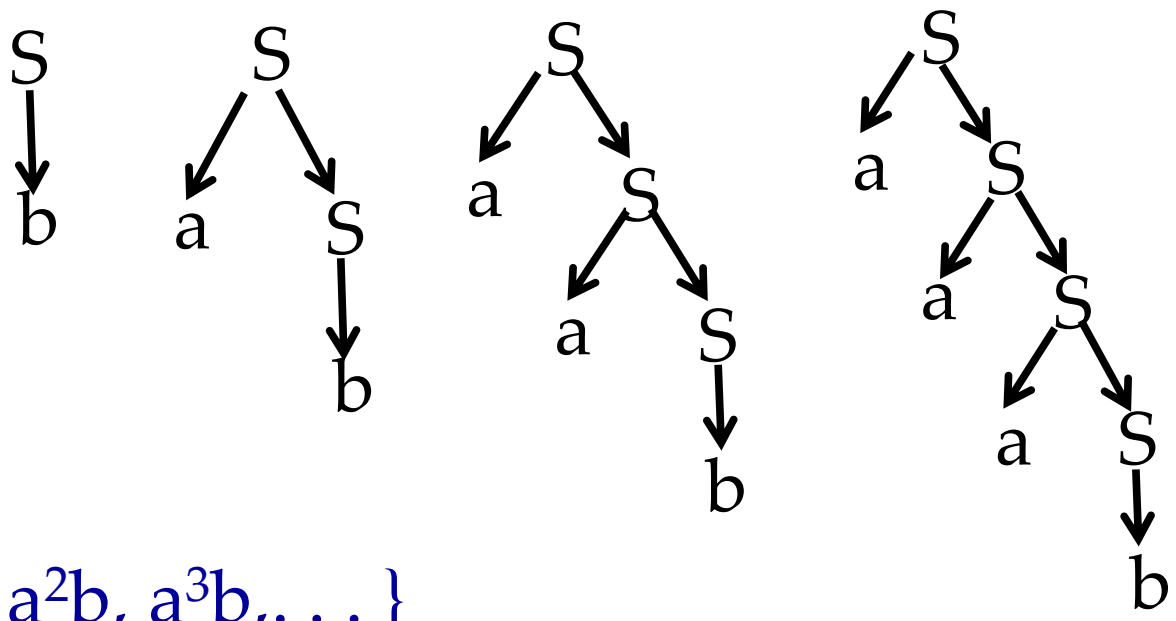
L(G) = { ab }

Grammar

G = ({S}, {a, b}, P, S), where

P: $S \rightarrow aS$

$S \rightarrow b$



$L(G) = \{ b, ab, a^2b, a^3b, \ldots \}$

$L(G) = \{ a^nb \ / \ n \geq 0 \}$

Example problems on grammar

1. Given $\Sigma = \{a, b\}$ obtain Σ^*
   - Give an example of a finite language in Σ
   - Given $L = \{a^n b^n : n>=0\}$, check if strings aabb, abb are in language, L

- Soln:
  - $\Sigma^* = \{\varepsilon, a, b, ab, ba, \ldots.\}$
  - $\{a, aa, aab\}$ → Some example of a language in Σ
  - String: 'aabb'

    $L = \{\varepsilon, ab, aabb, aaabbb, \ldots\}$

    When $n = 2$, $L = a^2 b^2$ therefore, it is a string in L
  - String: 'abb'

    This is not a string in L. Since, there is no equal number of a's and b's

Problem 2

Given $u = a^2ba^3b^2$; $v = ba$. Find (i) uv (ii) uv^2 (iii) $|uv|$

Soln:

(i) $uv = (a^2ba^3b^2).(ba) = a^2ba^3b^3a$

(ii) $uv^2 = (a^2ba^3b^2).(ba).(ba) = a^2ba^3b^3aba$

(iii) $|uv| = |a^2ba^3b^3a| = |aabaaabbba| = 10$

Exercise

- Consider the following two languages:

$L_1 = \{aa, bb\}$ where $\Sigma = \{a, b\}$

$L_2 = \{ppp, qqq\}$ where $\Sigma = \{p, q\}$

Compute the following:

a) $L_3 = (L_1)^2 . L_2$
b) $L_4 = L_2 . \varepsilon$
c) $L_5 = L_2 - L_3$
d) $L_6 = L_2 \cap L_4$
e) $L_7 = (L_5)^*$

Solution:

$L_3 = \{aaaa, aabb, bbaa, bbbb\} . \{ppp, qqq\}$

$L_4 = \{ppp, qqq\} . \varepsilon = L_2$

$L_5 = L_2 = \{ppp, qqq\}$

$L_6 = \{ppp, qqq\}$

$L_7 = \{\varepsilon, ppp, qqq, pppppp, qqqqqq, \ldots.\}$

Problem – Construct Language from Grammar

- Given a grammar, G = [{S}, {a, b}, S, P] where Production, P is given by S → a S b, S → ε. Obtain language L(G).

Soln:

S ==> a S b

==> a a S b b

==> a a a S b b b

==>….

==> a^n S b^n

==> a^n b^n

L (G) = {a^n b^n |where n≥0} or

L(G) = {set of strings with *n* a's followed by *n* b's |where n≥0}

Problem 1

- Given a grammar, G = [{A, S}, {a, b}, A, P] where Production, P is given by A → a S b, S → S b | ε. Obtain language L(G).

Soln:

A ==> a S b

==> a S b b

==> a S b b b

==> ….

==> a S b^n

==> a b^n

L(G) = { a b^n | where n≥1} or

L(G) = { All strings with exactly *one* 'a' and *n* b's}

Problem 2

G= ({S, B}, {a, b, c }, P, S), where

P: $S \rightarrow aSBc$ (rule 1)

$S \rightarrow abc$ (rule 2)

$cB \rightarrow Bc$ (rule 3)

$bB \rightarrow bb$ (rule 4)

S → **abc**

S → aSBc → aabcBc → aabBcc → **aabbcc**

S → aSBc → aaSBcBc → aaabcBcBc → aaabBccBc → aaabbccBc → aaabbcBcc → aaabbBccc → **aaabbbccc**

$L(G) = \{ a^n b^n c^n / n \geq 1 \}$

Problem 3

G= ({S}, {a, b, c }, P, S), where

P: $S \rightarrow aSa$ (rule 1)

$S \rightarrow bSb$ (rule 2)

$S \rightarrow c$ (rule 3)

$S \rightarrow c$

$S \rightarrow aSa \rightarrow$ **aca**

$S \rightarrow bSb \rightarrow$ **bcb**

$S \rightarrow aSa \rightarrow abSba \rightarrow$ **abcba**

$S \rightarrow bSb \rightarrow baSab \rightarrow$ **bacab**

$L(G) = \{ wcw^R / w \in \{a, b\}^* \}$

Construction of grammar

1) $L(G) = \{ a^n c a^n \ / \ n \geq 0 \}$

G= ({S}, {a, c}, P, S), where

P: $S \rightarrow aSa$

$S \rightarrow c$

2) $L(G) = \{ a^n b^{2n} \ / \ n \geq 0 \}$

G= ({S}, {a, b}, P, S), where

P: $S \rightarrow aSbb$

$S \rightarrow \varepsilon$

OR

P: $S \rightarrow aSA$, $A \rightarrow bb$

$S \rightarrow \varepsilon$

3) $L(G) = \{ a^{n+2} b^n \ / \ n \geq 1 \}$

G= ({S}, {a, b}, P, S), where

P: $S \rightarrow aSb$

$S \rightarrow aaab$

Problem 4

- Find the grammar for $\Sigma = \{0, 1\}$ that generate the set of all strings containing more 0's than 1's where 0's followed by 1's.
- Soln:

$A \rightarrow 0\ S$

$S \rightarrow 0\ S\ 1 \mid \varepsilon$

$G=(\{A,S\},\{0,1\},A,P)$

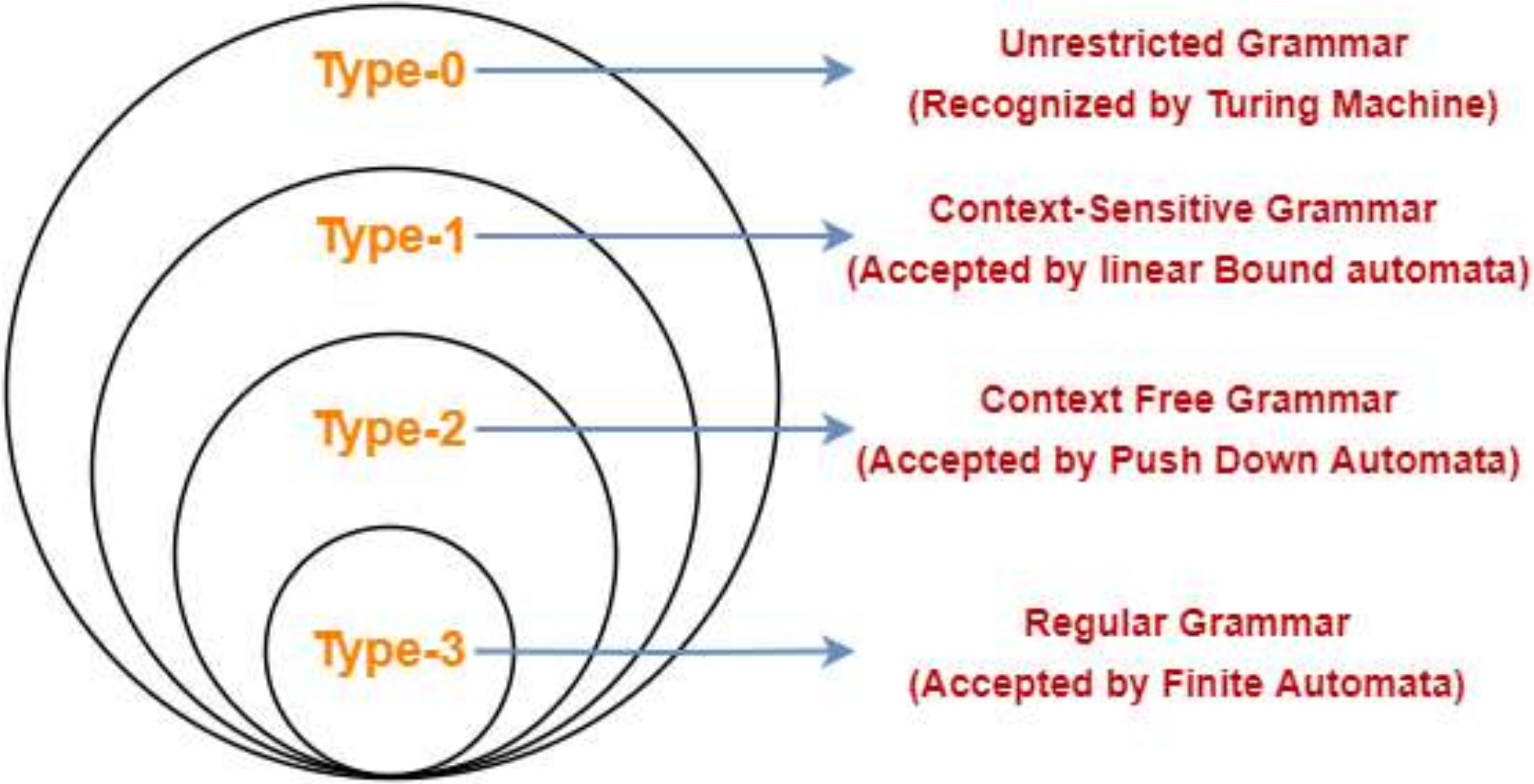
Practice problems

1) $L(G) = \{ a^n b^{n-3} \ / \ n \geq 3 \}$

2) $L(G) = \{ a^n b^m \ / \ n \geq 0, m > n \}$

3) $L(G) = \{ a^n b^n c^m \ / \ n, m \geq 1 \}$

Chomsky Hierarchy – Types of grammar



Chomsky Hierarchy

	Language	Grammar	Machine
Type 3	Regular languages	Regular grammars • Right-linear grammars • Left-linear grammars	Finite-state automata
Type 2	Context-free languages	Context-free grammars	Push-down automata
Type 1	Context-sensitive languages	Context-sensitive grammars	Linear-bound automata
Type 0	Recursive languages Recursively enumerable languages	Unrestricted grammars	Turing machines

Introduction to Formal Proof

- Types of Proof
  - Deductive proof:
    - A sequence of justified steps
  - Inductive proof:
    - Recursive proofs of a parameterized statements

Deductive Proofs

- Consists of a sequence of statements whose truth lead us from some initial statement called the hypothesis / given statement to a conclusion statement
- Hypothesis may be true / false, typically depending on the values of its parameters
- When we go from hypothesis H to conclusion C
  - IF H THEN C ➔ C is deduced from H
- Ex:
  - If $x >= 4$, then $2^x >= x^2$
  - If x is the sum of squares of four positive integers, then $2^x >= x^2$

Additional Forms of Proof

1. Proving Equivalence about sets

2. Proofs by contradiction

3. Proofs by counter example

1. Proving equivalence about sets

- If E and F are two expressions representing sets, the statement E = F means that the two sets represented are the same.
- We can prove the equivalence of Sets E = F in the following ways:
  1. By commutative law of union of two sets R and S as R U S = S U R

  Where E is the expression for R U S and F is the expression for S U R

  So, E = F by commutative law of union

Cont..

2. We can prove the set equality E = F by if and only if statement, that is "an element 'x' is in E iff 'x' is in F"

3. The set equality E = F can be proved by two if statements as follows:
   i. If 'x' is in E, then 'x' is in F
   ii. If 'x' is in F, then 'x' is in E

Contrapositive

- The contrapositive of the statement "if H then C" is "if not C then not H", and it is represented as

$$H \rightarrow C \Leftrightarrow 7C \rightarrow 7H$$

- To prove that "if H then C" and "if not C, then not H" are logically equivalent, there are four cases to consider:
  - H and C both are True
  - H is True and C is False
  - C is True and H is False
  - H and C both are False

Example

- "If 'x' is in E then 'x' is in F" and the contrapositive statement is

"if 'x' is not in F then 'x' is not in E"

2. Proof by contradiction

- Another way to prove the statement "if H then C" by the form,

H and not C implies contradiction

- Proof by Contradiction:
  - We assume that the Hypothesis and the negation of the conclusion as true and then we completed the proof by showing that something known to be false.

Example

- Prove the following statement by contradiction:

  The sum of two even numbers is always even.

- Soln:
  - Negate original statement:

    The sum of two even numbers is not always even.

  Take two even numbers, say 2 and 4, sum them (2 + 4 = 6) which is an even.

  Take two even numbers, say 12 and 16, sum them (12 + 16 = 30) which is an even.

  Therefore, by contradiction, the sum of two even numbers is always even

3. Counter Examples

- To prove theorem itself is true or false
- To solve this problem, we try to prove the theorem else if we are unable to prove the theorem, then we try to prove that the theorem statement is False.
- It is easy to prove that a statement is not a theorem than to prove it is a theorem.

Example:

- Theorem: All primes are odd

  Proof: if x is prime, then x is odd

  Disproof: if 2 is prime, then 2 is even [Not odd]

- Theorem: There is no pairs of integers ‘a’ and ‘b’ such that “a mod b = b mod a”

  Proof: If a and b are integers, then a mod b ≠ b mod a

  Disproof: Let a = b =2, then a mod b = b mod a ➔ 2 mod 2 = 2 mod 2 = 0

Inductive Proofs

- Deals with recursively defined objects like trees and expressions of various kinds such as regular expression
- Inductive proof includes the following:
  a. Induction on Integers
  b. Structural Induction
  c. Mutual Induction

a. Induction on Integers

- Suppose if we are given a statement, S(n) about an integer ‘n’ which is to be proved, then we have to prove the following:
  - Basis: We prove S(n) for particular integer ‘n’ when n = 0,1,2
  - Induction: If S(n) then S(n+1), when n>=i, where ‘i’ is basis integer
- General form of induction: S(i) S(i+1) S(i+2) …… S(n)
- Induction Principle:
  - If we prove S(i) and we prove that for all n>=i, S(n) implies S(n+1) then we conclude S(n) for all n>=i

Proof by mathematical induction

Proof by mathematical induction consists of three basic steps. If the statement **S** is to be proved then:

1) Show that **S** is true for some particular integer n_0

   - this is called **Basis**

2) Assume **S** is true for some particular integer $k \geq n_0$

   -this is called **Induction hypothesis**

3) Then to prove **S** is true for $k+1$

   - this is called **Induction step**

Example: 1

Show that for any $n \geq 1$, $1 + 2 + \ldots + n = n(n+1)/2$ by induction

Solution:

Let S(n) : $1 + 2 + \ldots + n = n(n+1)/2$

Basis step: S(1) : $1 = 1(1+1)/2 = 1$

S(1) is true.

S(2) : $1+2 = 2(2+1)/2 = 3$

S(2) is true

S(3) : $1+2+3$

$= 3(3+1)/2 = 6$

S(3) is true

Induction hypothesis: Assume that S(n) is true for some n.

S(n): $1 + 2 + \ldots + n = n(n+1)/2$ ----------(1)

Induction step: To prove S(n+1) is also true.

Take the LHS S(n+1) and prove the RHS.

$1 + 2 + \ldots + n + (n+1) = n(n+1)/2 + (n+1) = (n+1)\,(n/2 + 1)$ using (1)

$= (n+1)(n+2)/2$

Therefore, S(n+1) is true and hence S(n) is true for any n.

Problem 2

Prove the following by induction for all n >= 0,
$1^2 + 2^2 + \ldots + n^2 = n(n+1)(2n+1)/6$

Proof:

For $n = 1$, the statement reduces to $1^2 = \frac{1 \cdot 2 \cdot 3}{6}$ and is obviously true.

Assuming the statement is true for $n = k$:

$$1^2 + 2^2 + 3^2 + \cdots + k^2 = \frac{k(k+1)(2k+1)}{6}, \tag{1}$$

we will prove that the statement must be true for $n = k + 1$:

$$1^2 + 2^2 + 3^2 + \cdots + (k+1)^2 = \frac{(k+1)(k+2)(2k+3)}{6}. \tag{2}$$

The left-hand side of (2) can be written as

$$1^2 + 2^2 + 3^2 + \cdots + k^2 + (k+1)^2.$$

In view of (1), this simplifies to:

$$\begin{aligned}
\left(1^2 + 2^2 + 3^2 + \cdots + k^2\right) + (k+1)^2 &= \frac{k(k+1)(2k+1)}{6} + (k+1)^2 \\
&= \frac{k(k+1)(2k+1) + 6(k+1)^2}{6} \\
&= \frac{(k+1)[k(2k+1) + 6(k+1)]}{6} \\
&= \frac{(k+1)(2k^2 + 7k + 6)}{6} \\
&= \frac{(k+1)(k+2)(2k+3)}{6}.
\end{aligned}$$

Problem 3

Prove $1^3 + 2^3 + \ldots + n^3 = n^2(n+1)^2/4$ by mathematical induction for $n>=0$.

Proof:

For $n = 1$, the statement reduces to $1^3 = \frac{1^2 \cdot 2^2}{4}$ and is obviously true.

Assuming the statement is true for $n = k$:

$$1^3 + 2^3 + 3^3 + \cdots + k^3 = \frac{k^2(k+1)^2}{4}, \tag{5}$$

we will prove that the statement must be true for $n = k + 1$:

$$1^3 + 2^3 + 3^3 + \cdots + (k+1)^3 = \frac{(k+1)^2(k+2)^2}{4}. \tag{6}$$

The left-hand side of (6) can be written as

$$1^3 + 2^3 + 3^3 + \cdots + k^3 + (k+1)^3.$$

In view of (5), this simplifies to:

$$\begin{aligned}
\left(1^3 + 2^3 + 3^3 + \cdots + k^3\right) + (k+1)^3 &= \frac{k^2(k+1)^2}{4} + (k+1)^3 \\
&= \frac{k^2(k+1)^2 + 4(k+1)^3}{4} \\
&= \frac{(k+1)^2[k^2 + 4(k+1)]}{4} \\
&= \frac{(k+1)^2(k+2)^2}{4}.
\end{aligned}$$

Problem 4 Prove that $S(n) = a^n - b^n$ is divisible by (a-b) for all n>0

Let P(n) denote the statement $a^n - b^n$ is divisible by a – b.

Put n = 1. Then P(1) is the statement: $a^1 - b^1 = a - b$ is divisible by a – b

∴ P(1) is true. Now assume that the statement be true for n = k

(i.e.,) assume P(k) be true, (i.e.,) $a^k - b^k$ is divisible by (a – b) be true.

$$\Rightarrow \frac{a^k - b^k}{a - b} = m \text{ (say) where } m \in N$$

$$\Rightarrow a^k - b^k = m(a - b)$$

$$\Rightarrow a^k = b^k + m(a - b) \text{ ..... (1)}$$

Now to prove P(k + 1) is true, (i.e.,) to prove: $a^{k+1} - b^{k+1}$ is divisible by a – b

Consider $a^{k+1} - b^{k+1} = a^k . a - b^k . b$

$= [b^k + m(a - b)] a - b^k . b$ [$\because a^k = bm + k(a - b)$]

$= b^k . a + am(a - b) - b^k . b$

$= b^k . a - b^k . b + am(a - b)$

$= b^k(a - b) + am(a - b)$

$= (a - b) (b^k + am)$ is divisible by (a – b)

$\therefore$ P(k + 1) is true.

By the principle of Mathematical induction. P(n) is true for all $n \in N$.

$\therefore a^n - b^n$ is divisible by a – b for $n \in N$.

b) Structural Induction

- Deals with recursive definition that has

1. Basis: Where one or more elementary structures are defined

2. Induction: Where more complex structures are defined in terms of previously defined structures.

Example 1.19: Here is the recursive definition of a tree:

BASIS: A single node is a tree, and that node is the *root* of the tree.

INDUCTION: If $T_1, T_2, \ldots, T_k$ are trees, then we can form a new tree as follows:

1. Begin with a new node N, which is the root of the tree.
2. Add copies of all the trees $T_1, T_2, \ldots, T_k$.
3. Add edges from node N to the roots of each of the trees $T_1, T_2, \ldots, T_k$.

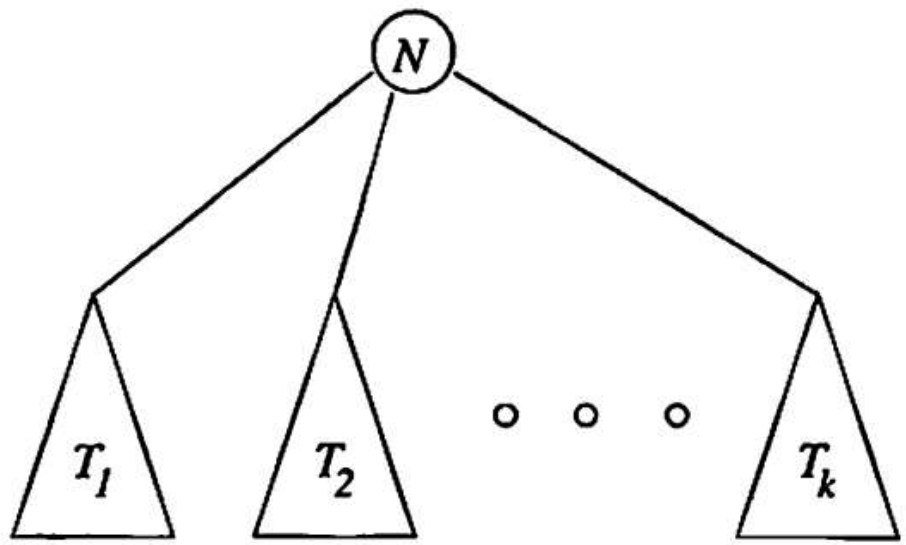


Figure 1.7: Inductive construction of a tree

c) Mutual Inductions

When there are several independent statements to prove then we keep the statements separate and prove them all in their own parts of the basis and inductive step.

Sometimes, we cannot prove a single statement by induction, but rather need to prove a group of statements $S_1(n), S_2(n),....,S_k(n)$ together by induction on 'n'